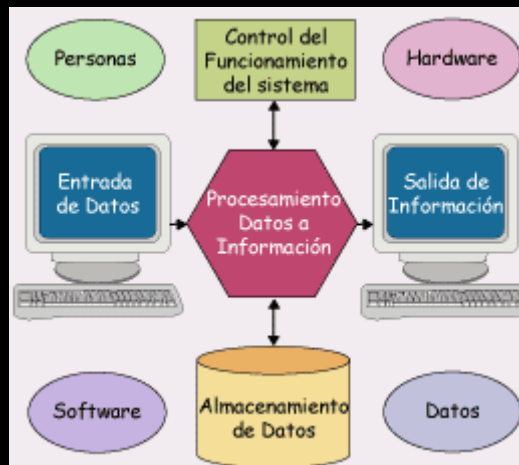


IF = información - ing = ingeniería del software - sw = software req = requerimientos - hw = hardware

Tema I - Sistemas de Información

Los sistemas de información:

Elementos de un sistema de IF



Sistemas Socio Tecnicos:

Los sistemas que incluyen sw se dividen en 2 categorías

Sistemas Tecnicos	Sistemas Socio - Técnicos
Sistemas que incluyen hw y sw pero NO procedimientos y procesos. Los individuos y organizaciones lo usan para un fin. Ej.: sw de aplicación, tv o teléfonos móviles	Comprenden 1 o más sistemas técnicos, pero también las personas incluyen conocimiento de cómo usarlo para sacar un objetivo más amplio (sacarle el jugo). definen procesos operativos, son gobernados por políticas/reglas gubernamentales. El enfoque es Industrial

Características de los Sistemas socio tecnicos (3):

1. Tienen propiedades Emergentes que son del sistemas englobados como un todo más que asociado con partes individuales. Dependen tanto de los componentes como de las relaciones entre los mismos para que funcionen exitosamente.
2. Son NO deterministas: Con una entrada específica no siempre se produce la misma salida. El comportamiento dependerá de operadores humanos
3. El grado en cual el sistema apoya a los objetivos no depende del mismo, también de la estabilidad, relaciones y conflictos de dichos objetivos.

La ing, es por lo tanto una crítica constructiva para el desarrollo acertado de sistemas complejos, el ingeniero no solo se debe preocupar del sw en sí mismo, sino también de cómo va a interactuar con el sistema de hw y demás sistemas involucrados.

Propiedades emergentes de los sistemas (2):

1. Propiedades emergentes funcionales: Cuando todas las partes del sistema trabaja en forma conjunta para lograr el objetivo.
2. Propiedades emergentes NO funcionales: Se refiere al comportamiento del sistema en su entorno operativo. como la fiabilidad, el rendimiento y la seguridad. (Relación operador y máquina)

IF = información - ing = ingeniería del software - sw = software req = requerimientos - hw = hardware

Impacto Etico y Social de los Sistemas de Información:

Tanto para los individuos como para las sociedades es fuente de muchos beneficios las tecnologías y los sistemas de IF. sin embargo crea nuevas oportunidades para violentar nuestra privacidad, siendo así que la protección del mismo se ha convertido en un gran problema ético y se debe tener presente los sig retos. (2)

1. Entender los riesgos morales de la nueva tecnología. Es importante que la gerencia realice un análisis de impacto ético y social de las nuevas tecnologías.
2. Establecer políticas de ética corporativa que incluyan cuestiones de sistemas de IF. Es evidente que las corporaciones debían contar con mecanismos para proteger la privacidad en internet y establecer derechos de IF siendo estas éticas la que el uso generalizado de Sistemas de IF dio origen.

Las Dimensiones Morales de los Sistemas de Información (5) - Políticas de conducta para cada dimensión

1. **Derechos de IF:** Estos sistemas amenazan con la privacidad de los individuos. Por ello se deben tomar cuestiones éticas, sociales y políticas para la privacidad y libertad en una sociedad de la IF.
2. **Derechos de Propiedad:** La propiedad intelectual se considera una propiedad intangible, creada por corporaciones que está sujeta a protecciones bajo las leyes. Se refiere a lo que es el derecho de autor y patentes
3. **Responsabilidad formal y de control:** Habla de la responsabilidad por los daños a los derechos de IF
4. **Calidad del Sistema:** Existe una barrera tecnológica que impide llegar al sw perfecto, entonces los usuarios deben ser conscientes de la posibilidad del mismo debido a la calidad de los datos.
5. **Calidad de vida:** Si bien mejora la calidad de vida, el usuario debe ser responsable del uso ético de los sistemas de IF y de las consecuencias que trae el abuso del mismo, ya que todo en exceso es malo. por ejemplo las enfermedades causadas por computadoras.

Para poder dar una clasificación de los sistemas de IF, debemos saber que existen 2 tipos de Organizaciones: Estas Pueden ser 1) **Tradicionales** (hasta 1990) - 2) **Inteligentes** (actualidad) -> si hablamos de tipos de sistemas de IF en la teoría se refiere a las organizaciones tradicionales.

Sistema de procesamiento de transacciones (TPS): gestiona la información referente a las transacciones producidas en una empresa u organización, también se le conoce como Sistema de Información operativa.

Sistemas de información ejecutiva (EIS): herramienta orientada a usuarios de nivel gerencial, que permite monitorizar el estado de las variables de un área o unidad de la empresa a partir de información interna y externa a la misma. Es en este nivel cuando los sistemas de información manejan información estratégica para las empresas.

Sistemas de información gerencial (MIS): orientados a solucionar problemas empresariales en general.

Sistemas de soporte a decisiones (DSS): herramienta para realizar el análisis de las diferentes variables de negocio con la finalidad de apoyar el proceso de toma de decisiones.

Sistema experto (SE): emulan el comportamiento de un experto en un dominio concreto. **Sistema Planificación de Recursos (ERP, Enterprise Resource Planning):** cuyo objetivo es la planificación de los recursos de una organización.

Sistemas de automatización de oficinas (OAS): aplicaciones destinadas a ayudar al trabajo diario del administrativo de una organización.

Sistemas de IF Estratégicos (Enfocado a Organizaciones Inteligentes)

Puede ser considerado como el uso de la tecnología de la información para respaldar o dar forma a la estrategia competitiva de la organización, a su plan para incrementar o mantener la ventaja competitiva o bien para reducir la ventaja de sus competidores.

IF = información - ing = ingeniería del software - sw = software req = requerimientos - hw = hardware

Tema II - Gestión de Proyectos

Ingeniería del Software:

La ingeniería del software es el establecimiento y uso de principios de ingeniería robustos, orientados a obtener económicamente software que sea fiable y funcione eficientemente sobre máquinas reales. Bauer F.

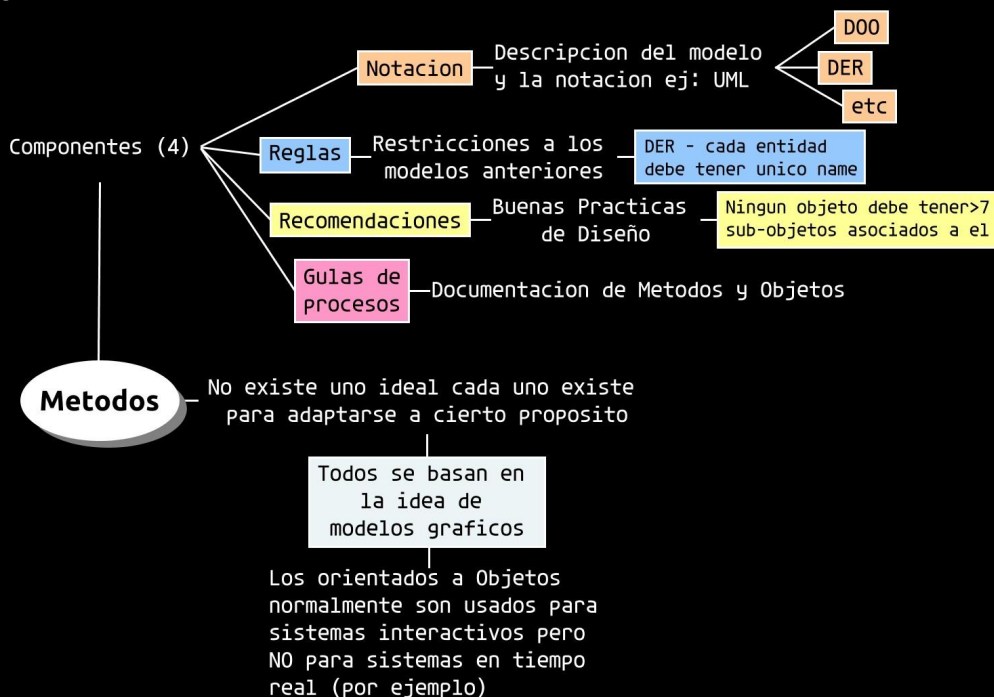
una disciplina de ingeniería que comprende todos los aspectos de la producción del sw.

Ingeniería del Software	Ciencia de la Computación
Comprende las formas prácticas para desarrollar y entregar un sistema útil	Comprende la teoría y los Fundamentos
Ingeniería del Software	Ingeniería de Sistemas
Es una parte o un subsistema de todo lo que abarca el proceso de ingeniería de sistemas	Se refiere a todos los aspectos del desarrollo de sistemas informáticos, incluyendo hardware, software e ingeniería de procesos

Retos fundamentales a los que se enfrenta la ing:

- 1) Heterogeneidad: A menudo es necesario integrar software nuevo que funcione con un sistema ya heredado escrito en otro lenguaje de programación, y se operen como sistemas distribuidos en red.
- 2) Entrega: El gran tema es reducir los tiempos de entrega para sistemas grandes y complejos sin que pueda ser afectada la calidad del mismo.
- 3) Confianza: Desarrollar técnicas donde los usuarios puedan confiar en él y no transmita desconfianza.

Métodos en la Ingeniería de Software:



Atributos y Características del Software:

Para poder comprender lo que es el sw y la ing en la teoría da una diferencia con respecto al hardware y el sw, fundamentando que el sw al ser un elemento del sistema lógico y NO físico va a tener las siguientes características considerables:

IF = información - ing = ingeniería del software - sw = software req = requerimientos - hw = hardware

1. El sw de desarrolla: Aunque en este proceso existen similitudes con el hardware el enfoque es diferente, por este motivo el sw NO se puede gestionar como si fuera un proceso de fabricación ya que los costes se encuentran en la ingeniería.
2. El sw no se estropea: En el primer ciclo de vida es inevitable que tenga defectos no detectados, sin embargo una vez corregidos la curva se aplanan considerablemente, en etapa de mantenimiento cuando se agregan nuevas funcionalidades podría aparecer algunos picos en la curva hasta que sean parchados, indicando que el mantenimiento tiene una complejidad considerable mayor a la del hardware.
3. Aunque la industria tiende a ensamblar componentes, la mayoría del sw se construye a medida. Así como en el hardware la reutilización de piezas estándar ya es natural y no novedosa, en el sw todavía no está del todo logrado, sin embargo hay muchas piezas de software reutilizables para construir nuevos sistemas a través de ya existentes. Ejemplo: login o las Interfaces gráficas de usuario

La complejidad inherente del Software:

El software es un transformador de información, produciendo, gestionando, adquiriendo, modificando, mostrando o transmitiendo información que puede ser tan simple como un solo bit, o tan complejo como una presentación en multimedia.

Gestión de Proyectos:

Una buena gestión NO puede garantizar el éxito del proyecto, sin embargo la mala gestión usualmente lleva al fracaso del proyecto. El sw se entrega tarde, los costes son mayores que los estimados y los req NO se cumplen. Los gestores del sw son los responsables de la calendarización y planificación del desarrollo del proyecto. Esto es necesario ya que la ing está sujeta a restricciones de tiempo, presupuesto y estándares req.

Planificación del Proyecto:

La gestión efectiva de un proyecto dependerá de planificar *completamente* el progreso del proyecto. El plan preparado al inicio, además de utilizarse como conductor del proyecto también tiene que ser el mejor posible en cuanto a la IF que se obtenga disponible. A medida que el mismo progrese evoluciona y la IF será mejor.

- restricciones del proyecto: personas, presupuesto, tiempo acordado, hw y sw, etc.

```
Establecer las restricciones del proyecto
Hacer la valoración inicial de los parámetros del proyecto
Definir los hitos del proyecto y productos a entregar
Mientras el proyecto no se haya completado o cancelado repetir
    Bosquejar la programación en el tiempo del proyecto
    Iniciar actividades acordes con la programación
    Esperar (por un momento)
    Revisar el progreso del proyecto
    Revisar las estimaciones de los parámetros del proyecto
    Actualizar la programación del proyecto
    Renegociar las restricciones del proyecto y los productos a entregar
    Si (surgen problemas) entonces
        Iniciar la revisión técnica y la posible revisión
    fin de si
fin de repetir
```

El proceso de planificación del proyecto es un proceso iterativo que solo se completa cuando el ciclo termina o se cancela. Conforme la IF se hace disponible el plan se va revisando constantemente. Ahora cuando ya se tenga suficiente IF los gestores revisan sus hipótesis y la agenda, si se percatan que el proyecto se va a retrasar tienen que renegociar con el cliente la entrega del mismo, Si no se tiene éxito tendrán que buscar otra alternativa que cumpla las metas del calendario. (Muchos autores recomiendan que la negociación no pase)

- El Plan del Proyecto: Es un documento que fija los recursos disponibles, divide el trabajo y crea el calendario de trabajo. Conforme pasa el tiempo pasa lo mismo que lo anterior: algunas partes van a tener que cambiarse, como por ejemplo la calendarización. Para agilizar este paso muchos autores recomiendan estructurar este documento en secciones (IEEE 830).

IF = información - ing = ingeniería del software - sw = software req = requerimientos - hw = hardware

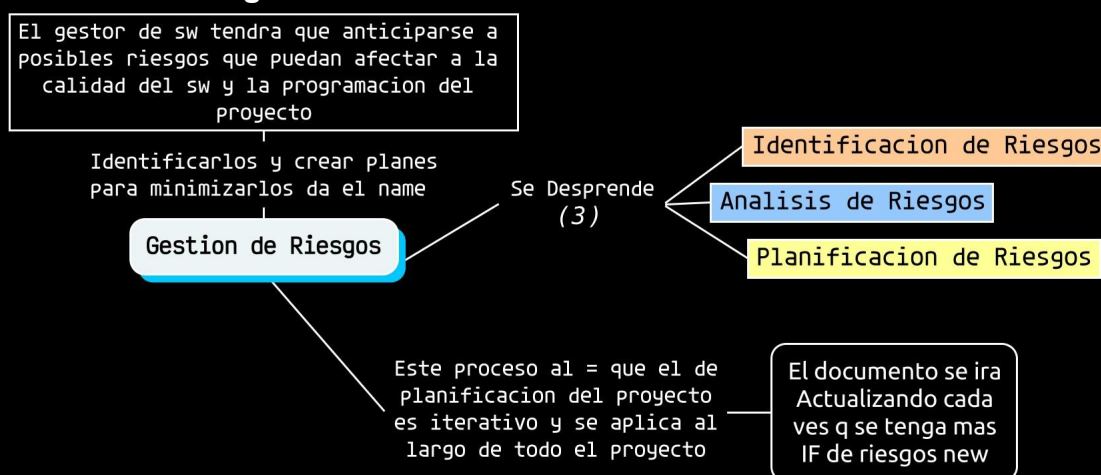
- **Hitos y Entregas:** Los hitos son terminaciones de cada actividad pequeña asignada, cuando finalizada se tendrá que entregar un informe corto del mismo para que el gestor lo lea.
Las entregas son los resultados que se entregan al cliente (literalmente).
También son considerados hitos sino que no ellos no se entregan al cliente, el gestor lo utiliza para verificar el final de la actividad, serían resultados internos del proyecto. Lo que sí se entregan al cliente son los manuales y el ERS.

Calendarización del Proyecto:

Es una de las tareas más difíciles para los gestores de proyectos. Los gestores estiman tiempo y los recursos necesarios requeridos para completar las actividades, al menos que el gestor de sw ya haya trabajado en un proyecto similar, las estimaciones previas son una base muy incierta por el hecho de que diferentes proyectos puedan usar métodos de diseño y lenguajes de programación diferentes.

Este proceso se basa en separar todo el trabajo del proyecto en actividades complementarias de forma óptima, también se debe tener en cuenta los recursos: Principalmente el esfuerzo humano y segundo el hardware y todo lo relacionado, visualmente se lo observa como diagrama de Gantt o de Pert.

Analisis y Gestion de Riesgos:



Sommerville clasifica 3 tipos de Riesgos:

Proyecto	Producto	Negocio
Afectan a la calendarización o los recursos del proyecto	Afectan a la calidad o al rendimiento del sw desarrollado	Afectan a la organización que suministra el software

#Dato: Normalmente un riesgo se clasifica en 2 o 3 tipos a la vez. Ej.: Programador experto se toma el palo.

Identificación de Riesgos:

Es la primera etapa de la gestión de riesgos, comprende el descubrimiento de los posibles riesgos del proyecto, en este proceso se puede utilizar brainstorming o ya los gestores se basan en la experiencia.

Análisis de Riesgos:

Se considera por separado cada riesgo de la etapa anterior y se le asigna una probabilidad y seriedad del mismo, por supuesto cuando se tenga más IF estos datos serán actualizados en el documento en cada iteración. Los datos a usarse para las probabilidades son cualitativos.

Planificación de Riesgos:

En esta etapa el gestor de sw crea planes para eliminar o al menos minimizar los riesgos. Esta tarea es muy compleja y relativa. Ya va a depender más de la experiencia que tenga el gestor.

Supervisión de Riesgos:

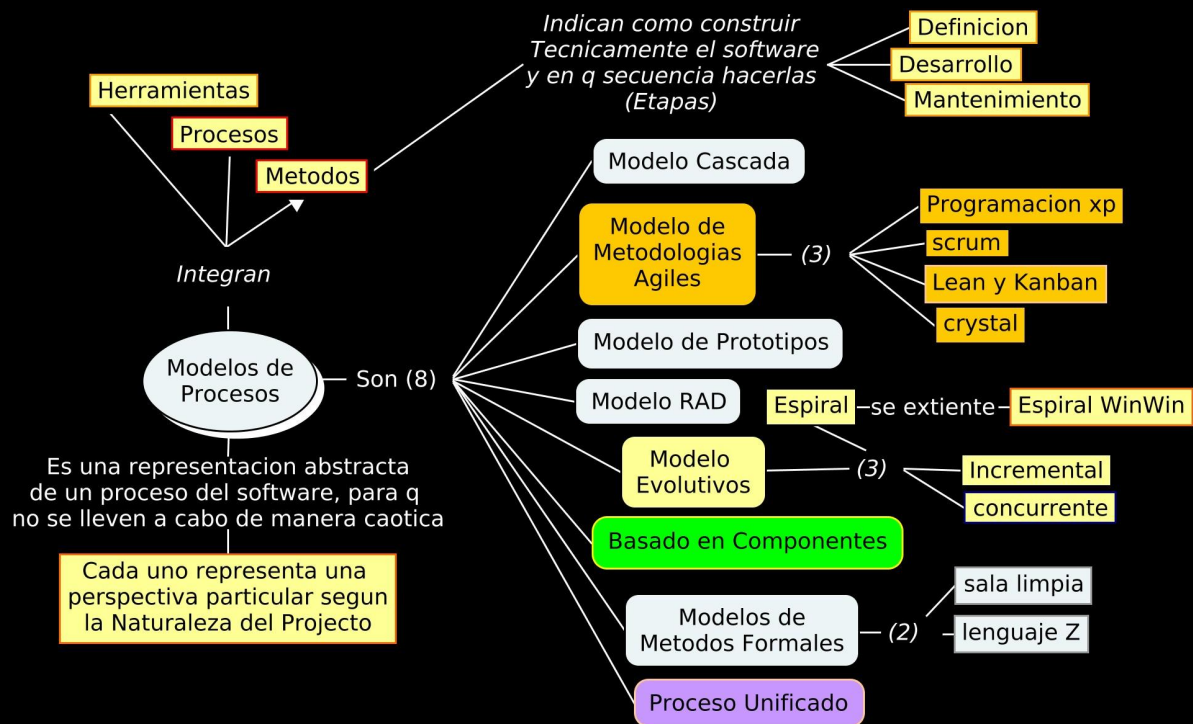
Valorar los riesgos de forma constante y revisar los planes para la mitigación de riesgos tan pronto como la IF de los riesgos esté disponible.

IF = información - ing = ingeniería del software - sw = software req = requerimientos - hw = hardware

Tema III - Procesos del Software

El proceso del Software es un conjunto de Actividades o marco de trabajo para un conjunto de áreas, las cuales conducen a la creación de un producto (software) y dichas actividades pueden consistir en el desarrollo desde cero.

Aunque no existen procesos del software "ideal", en las organizaciones existen enfoques para mejorarlos. De hecho, muchas organizaciones aún no aprovechan los métodos de la Ing en el desarrollo de su software.



1 **Modelo en Cascada:** Fue uno de los primeros procesos, se lo denomina como un proceso genérico, y tal como lo dice el nombre: va en cascada de una fase a otra. Siendo las fases actividades del desarrollo. Las fases son (5)

1. Definición de Requerimientos: Se incluye todo el documento (ERS) a partir de las consultas o interacciones con el usuario.
2. Diseño del Software: Establece una Arquitectura completa del sistema e identifica y describe las abstracciones.
3. Implementación y Prueba de Unidades: Se lleva a cabo el diseño del sw como un conjunto de unidades y se realizan las pruebas de aceptación.
4. Integración y Pruebas del Software: Se integran todas las unidades de la fase anterior y realizan las pruebas como un sistema completo para asegurarse que haga exactamente lo que pidió el cliente, finalizado el mismo se entrega el producto.
5. Funcionamiento y Mantenimiento: Se instala el sistema y se corrigen errores no descubiertos en las fases anteriores.

Ventaja: La documentación se produce en cada fase y este modelo se encuadra con los demás modelos.

Problemas (2) :

- **Interacciones:** Son costosas e implica rehacer trabajo, si son muchas se congelan partes del desarrollo
- **Mantenimiento:** Hacer cambios en esta etapa implica repetir las etapas previas del proceso

Raramente los proyectos siguen este ciclo de vida, el cliente pocas veces establece todos los requerimientos al principio, el cliente no tiene un producto terminado hasta el final, el tiempo de desarrollo es muy largo. Solo se debe utilizar cuando los req se comprendan bien al principio y no cambien radicalmente en la etapa de desarrollo.

IF = información - ing = ingeniería del software - sw = software req = requerimientos - hw = hardware

2 Modelo de Prototipos: Este modelo se utiliza para dar al usuario una vista preliminar de parte del sw. Básicamente es prueba y error ya que si al usuario no le gusta una parte del prototipo significa que la prueba falló implicando corregirlo hasta que el usuario quede satisfecho. Debe ser construido en poco tiempo y no debe utilizarse mucho dinero, porque a partir de que el mismo sea aprobado se iniciará el verdadero proyecto. Se lleva a cabo elaborando un prototipo del producto final, puede ser tan simple como dibujo a lápiz o tan complejo como código operativo final. Se hace mucho énfasis en las interfaces. con sw adecuado

Etapas:

1. Recolección y refinamiento de requisitos
2. Modelado, diseño rápido
3. Construcción del prototipo
4. Desarrollo: evaluación del prototipo del cliente
5. Refinamiento del prototipo
6. Producto de Ingeniería

Ventaja: Reduce Riesgos y costos de tal modo que aumenta la probabilidad del éxito.

Problemas: El cliente lo quiere, aunque no es un producto y módulos ineficientes se convierten en partes del sistema.

Este Modelo es adecuado usar cuando el cliente no puede especificar los requerimientos al principio, debe ser un sistema con el que se pueda experimentar siendo comparativamente barato y que sea desarrollado rápidamente. El prototipo puede ser (1) desechado y servirá para eliminar dudas de lo que el cliente quiere o bien (2) evolucionario pasando a ser un prototipo del software pero desfavorable con poca documentación.

3 Ingeniería de Software basado en Componentes: Se basa en la reutilización, el cual se está utilizando de forma amplia, sucede cuando personas que trabajan en el proyecto conocen diseños o códigos similares al requerido. El paradigma orientado a objetos enfatiza en la creación de clases que encapsulan tanto los datos como los algoritmos. Es evolutivo por naturaleza y exige un enfoque iterativo para la creación de sw. Se configuran las aplicaciones desde componentes preparados de software.

La reutilización según estudios establece que se reduce el ciclo de vida en un 70%, reduce el costo del proyecto en un 84% y aumenta la productividad. Tiene mucho foco en la integración de servicios web o SOS.

La única desventaja sería si la organización no tiene control de las nuevas versiones de los componentes.

De las (6) fases la 1ra y la 6ta son iguales a los demás modelos, solo interesan las 4 del medio



4 Modelo Desarrollo Rápido de Aplicaciones (RAD): Está basado en el modelo en cascada y el modelo de componentes de manera que: enfatiza un ciclo de vida corto (60 -90 días) se define como una adaptación de alta velocidad donde logra el desarrollo rápido utilizando componentes y herramientas CASE que generan código.

Ventajas: Si se entiende bien los requisitos y se limita el ámbito del proyecto, permite crear un sistema completamente funcional en poco tiempo, y también permite la reutilización al usarse componentes.

Problemas (4):

- **Recursos Humanos:** Es requerido para proyectos grandes y generar equipos de trabajo.
- **Compromiso:** Es requerido tanto los clientes como los desarrolladores, al contrario el proyecto fracasa
- **Riesgos:** No es adecuado cuando los riesgos técnicos son altos.
- **CASE:** El uso de estas herramientas obviamente tendrán un costo al igual que el equipo necesario.

No todos los tipos de aplicación son apropiados para este modelo, Si un sistema no puede modularizar adecuadamente la construcción de componentes sería caótico, Tampoco si está en juego en los req un alto rendimiento ya que en este modelo puede fracasar, de manera contraria si un sistema puede modularse de forma que permita terminar cada una de las funciones principales en menos de (90 días) este modelo si sería el candidato perfecto

IF = información - ing = ingeniería del software - sw = software req = requerimientos - hw = hardware

5 Modelo de Proceso Unificado: Este modelo es un proceso basado en componentes que ha sido propuesto por la industria, dispone de un Lenguaje de Modelado Unificado (UML), que es utilizado para construir el sistema y las interfaces que conectan los componentes. Combina un desarrollo incremental e iterativo, definiendo la función del sistema aplicando un enfoque basado en escenarios. Se puede decir que es:

1. **Dirigido por los casos de uso**: basándose en el modelo de casos de uso y en su análisis los desarrolladores crean los modelos de diseño e implementación que lleven a cabo los casos de uso.
2. **Centrado en la arquitectura**: la arquitectura de un sistema de software se describe mediante diferentes vistas del sistema de construcción
3. **Iterativo e Incremental**: el desarrollo de un proyecto se divide en iteraciones, encargadas de pequeñas partes del trabajo y que dan como resultado un crecimiento del producto, incremento. Estas iteraciones se planifican y se prueban cada vez que terminan.

6 Modelo de Métodos Formales: Comprende el conjunto de actividades que conducen a la especificación matemática del software de computadora, permite que un ingeniero especifique, desarrolle y verifique un sistema aplicando una notación rigurosa y matemática.

7 Modelos Evolutivos - Modelo Incremental: El modelo Incremental combina elementos del modelo lineal secuencial(aplicados repetidamente) con la filosofía interactiva de construcción de prototipos. Los clientes van a identificar a grandes rasgos los servicios que proporciona el sistema haciendo énfasis en cuales son más importantes y cuáles menos.

Ventajas: (Cliente) (3)

- Tiene una entrega temprana de la funcionalidad del sistema, ayudando a clarificar sus req y no esperan a q el sistema completo se entregue a ≠ del modelo secuencial.
- El primer incremento satisface los req más críticos, también puede utilizarlos como prototipos y obtener experiencia sobre los incrementos siguientes.
- El riesgo de un fallo es muy bajo = que el modelo de prototipos

Problemas: Resulta difícil adaptar req del cliente a un tamaño apropiado, ya que la idea es minimizarlos

Existe una variante a este Modelo denominada Programación Extrema (xp)

Modelos Evolutivos - Modelo Espiral: Es un modelo de proceso de software evolutivo que conjuga la naturaleza iterativa de construcción de prototipos con los aspectos controlados y sistemáticos del modelo lineal secuencial. Ideal para realizar versiones incrementales de manera rápida, el software se desarrolla en una serie de versiones incrementales. La diferencia principal entre los demás modelos es que se considera mucho el riesgo, es candidato para proyectos donde se requieran alto nivel de riesgo.

Espiral (Win Win): Se extiende del anterior como un conjunto de actividades de negociación al principio de cada paso alrededor de la espiral. Es decir se agrega la funcionalidad de comunicación con el cliente:

- en la comunicación ideal: el desarrollador pregunta al cliente y este le facilita suficiente información para continuar.
- en la comunicación real: el cliente y el desarrollador, entran en un proceso de negociación, donde el cliente puede ser preguntado para sopesar la funcionalidad, rendimiento y otras características.

Modelos Evolutivos - Desarrollo Concurrente: El modelo define una serie de acontecimientos que disparan transiciones de estado a estado para cada una de las actividades de la ingeniería del software. Este modelo se utiliza a menudo como el paradigma de desarrollo de aplicaciones cliente/servidor.

Ingeniería de Software Asistida por Computadora CASE: Nombre que se le da al software que se utiliza para ayudar a las actividades del proceso como la ingeniería de req, el diseño, el desarrollo y las pruebas. Permitiendo mejoras y calidad en la productividad del software a desarrollarse.

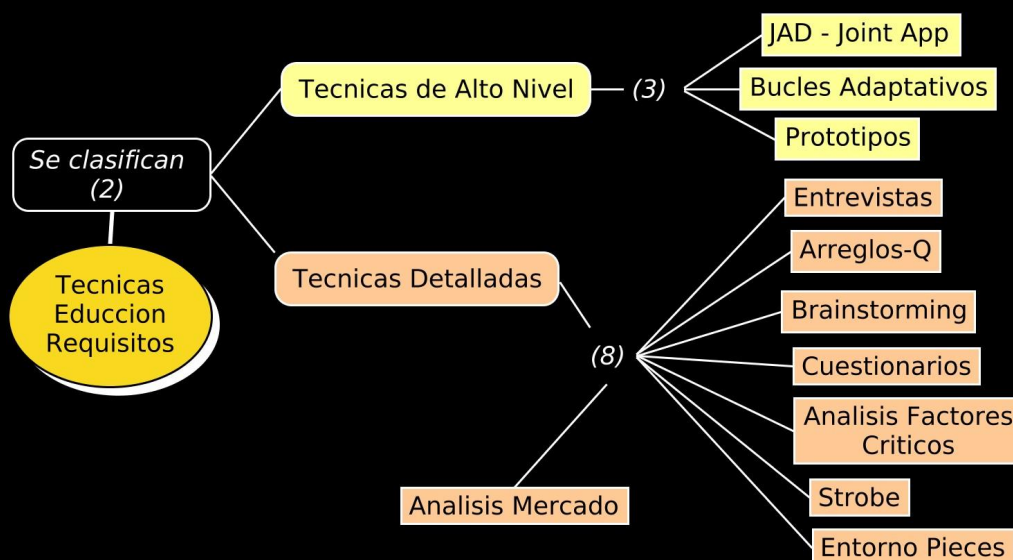
Tema IV - Requerimientos del Software

El proceso de Educación de Requisitos: puede descomponerse en las actividades de: hallazgo, de hechos, reunión e integración de la información. También se presentan problemas(3) en esta etapa como:

Problemas de Alcance: Los req pueden implicar demasiada o muy pobre IF.

Problemas de Comprensión: Es causado por una pobre comunicación entre el analista y el cliente, como consecuencia requisitos que no cumplen sus características ERP.

Problemas de Volatilidad: Req que evolucionan con el tiempo del cual se aleja al req inicial obtenido.



1) **Entrevistas:** Conversación dirigida por un propósito específico basada en un formato de Preguntas y Respuestas. Se desea conocer las opiniones como los sentimientos del entrevistado y metas personales. Las preguntas pueden ser: Abiertas, Cerradas o de Sondeo del cual organizará la estructura de la misma. En cuanto al registro de la misma, se puede usar (1)Grabadora o (2)Libreta de Notas c/u con pro y contras.

2) **Cuestionario:** Es un instrumento de investigación útil y eficaz para recoger IF en un intervalo de tiempo breve, esta técnica es la más usada y muchas veces la más factible a utilizarse. Sin embargo, elaborar un cuestionario no es una tarea nada fácil.

3) **Arreglos-Q:** Consiste en la Estructuración de un arreglo en el cual fuerza a que las respuestas se apeguen a una distribución Normal (Estadística). A partir de ello el analista crea una serie de tarjetas con postulados para que el receptor las ordene.

4) **Brainstorming - Lluvia de Ideas:** Es la aportación de ideas que varias personas ponen en común como punto de partida para un proyecto. Una sesión funciona bien entre 4-10 personas. Es útil para la generación de múltiples puntos de vista en cuanto al dominio del problema.

5) **Entorno Pieces:** Es poco usada por el grupo de analistas y consta de (6) categorías. 1)Rendimiento 2)IF y datos a obtener del sistema 3)Economía 4)Control 5)Eficiencia 6)Servicios por el Sistema.

6) **Análisis del Mercado:** Es una actividad común en la mayoría de empresas que venden productos porque necesitan conocer el mercado y por ende saber como funciona antes de construir del producto. Basándose por ej.: En la competencia o en los datos estadísticos y tendencias.

7) **JAD - Joint Application Design:** Diseño de sistemas basados en computadoras centrándose en el trabajo conjunto entre el *usuario que solicita el servicio* y el *analista*. promoviendo la comprensión, cooperación y el equipo de construcción obteniéndose una visión compartida de lo que el sistema debería ser

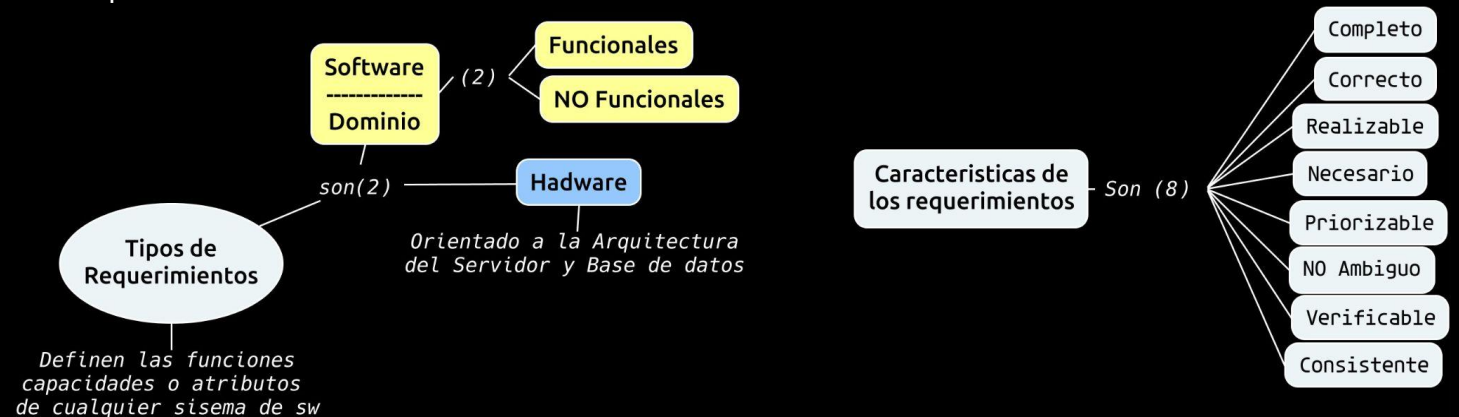
- Es candidato si el grupo es demasiado grande, caso contrario es muy costoso aparte de que carece de precisión si el proyecto es pequeño. (Está pensado para proyectos muy grandes).

8) **Entorno de Bucles Adaptativos:** Estrategia donde interactúa durante la etapa completa de ingeniería de requerimientos los: (3) 1-usuario 2-desarrollador 3-sistema.

IF = información - ing = ingeniería del software - sw = software req = requerimientos - hw = hardware

9) **Prototipos:** Si bien cuando hablamos del mismo ya estamos hablando de una metodología, esta permite identificar con rapidez las necesidades particulares de IF al usuario, en donde dichas necesidades se implementan rápidamente mediante iteraciones o proceso evolutivo en las cuales se refinan en cada iteración, sin embargo la administración del mismo llega a ser difícil como un proyecto.

10) **Observación al Comportamiento de la toma de decisiones y al ambiente de oficina STROBE:** Se hace uso de la observación para obtener IF de los tomadores de decisión y su ambiente, también auxilia a confirmar lo que las entrevistas y los cuestionarios hubieran detectado, o para rebatir o anular que otros métodos encontraron.



Requerimientos de usuario: Estos deben describir los req funcionales y no funcionales, pero de forma que sea comprensible para todos los usuarios en especial a los no técnicos, con diagramas intuitivos. Evitando lo máximo posible las características del sistema.

Requerimientos del sistema: Son versiones extendidas de los req del usuario, usados por los ing como punto de partida para el diseño del sistema. Aca si se explica detallada y técnicamente cómo se deben implementar los req del usuario usando el lenguaje técnico, debiendo describir el comportamiento externo del sistema y las restricciones del mismo. Se puede concluir lo siguiente: (req sistema $\subset$ (req usuario $\subset$ (req NO/SI funcional)))

Especificación de los Requerimientos: Se puede utilizar el lenguaje natural para redactar ambos req, sin embargo los req del sistema al ser más detallados pueden ser difíciles de entender o ser interpretadas diferente para ambos lectores resultando costoso poder resolverlos a futuro. Por ese motivo se recomienda complementarlo con casos de usos o lenguajes matemáticos formales.

Historias de Usuario:

Estudios de Viabilidad: Para todos los sistemas nuevos el proceso de ing de req debería empezar por un estudio de viabilidad. Se trata de una descripción resumida del sistema y de cómo va a contribuir a los procesos de negocio, el resultado es un informe en donde se va a recomendar (o NO) seguir con la ing de req, en dicho estudio se pueden consultar las distintas fuentes de IF que tenga a su disposición. Normalmente dura de 2-3 semanas.

Desarrollo de la ingeniería de requerimientos (4): Las mismas son iterativas

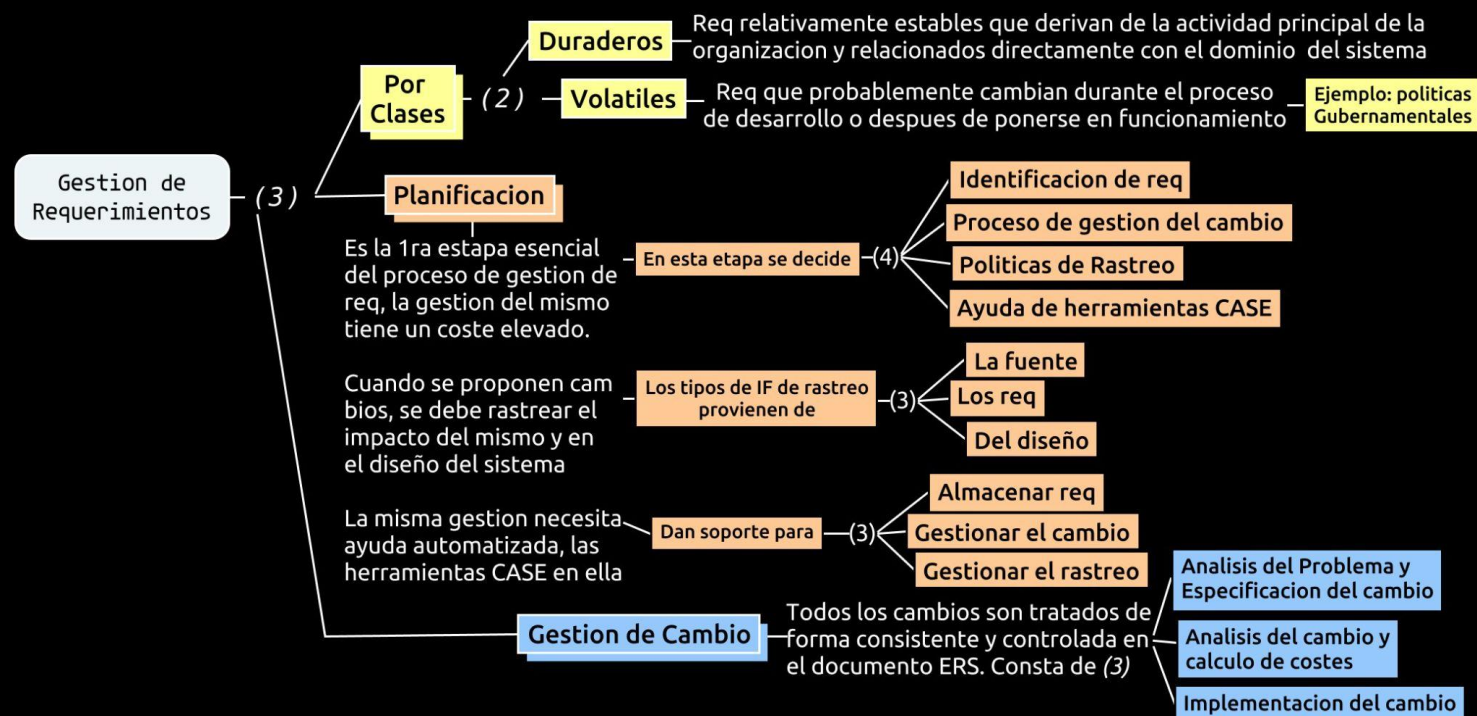
- ☐ **Recolección:** Proceso entre los clientes y el equipo de construcción en donde articulan las necesidades del sistema y las restricciones que tendrá el mismo.
- ☐ **Análisis:** Analizan las necesidades del cliente y los usuarios para llegar a 1 definición de los req del sw
- ☐ **Especificación:** Creación del documento ERS.
- ☐ **Verificación:** Se asegura que la ERS es una base sólida para el diseño y la arquitectura.

Validación de Requerimientos: Trata de mostrar que req realmente definen el sistema que el cliente desea. Coincide con el análisis ya que este implica encontrar problemas en los req. Es importante debido a que el coste de hacer un cambio en los req para arreglar un problema es mucho mayor que reparar errores de diseño/codificación. Durante el proceso de validación se debe verificar que los req cumplan las (8) características mencionadas anteriormente.

IF = información - ing = ingeniería del software - sw = software req = requerimientos - hw = hardware

#Dato importante es que rara vez encuentran todos los problemas en este proceso debido a la complejidad del mismo implicando cambios adicionales en el documento ERS.

Gestión de Requerimientos: Los req para sistemas grandes siempre son cambiantes. Una razón es que el problema no puede definirse completamente y por ende los req son incompletos, entonces estos mismos deben evolucionar para reflejar la perspectiva cambiante del problema. Además una vez instalado el sistema surgen nuevos cambios en los req ya que es muy difícil evitarlos. Hay que establecer un proceso formal para implementar las propuestas de cambios y vincularlos a los req del sistema.



Trazabilidad de Requerimientos: Es la capacidad de describir y de seguir la vida de un requisito, tanto en dirección hacia adelante y hacia atrás, es decir, desde sus orígenes, a través de su desarrollo y especificación, a su despliegue y uso subsecuentes, y a través de períodos de refinamiento y de la iteración en curso en cualesquiera de estas fases.

Permite ver dónde se originó un requerimiento. En otras palabras, todos los documentos de requerimientos, Los documentos de diseño, código y guiones de prueba deberán apuntar a su fuente de origen.

Tipos de Trazabilidad (3):

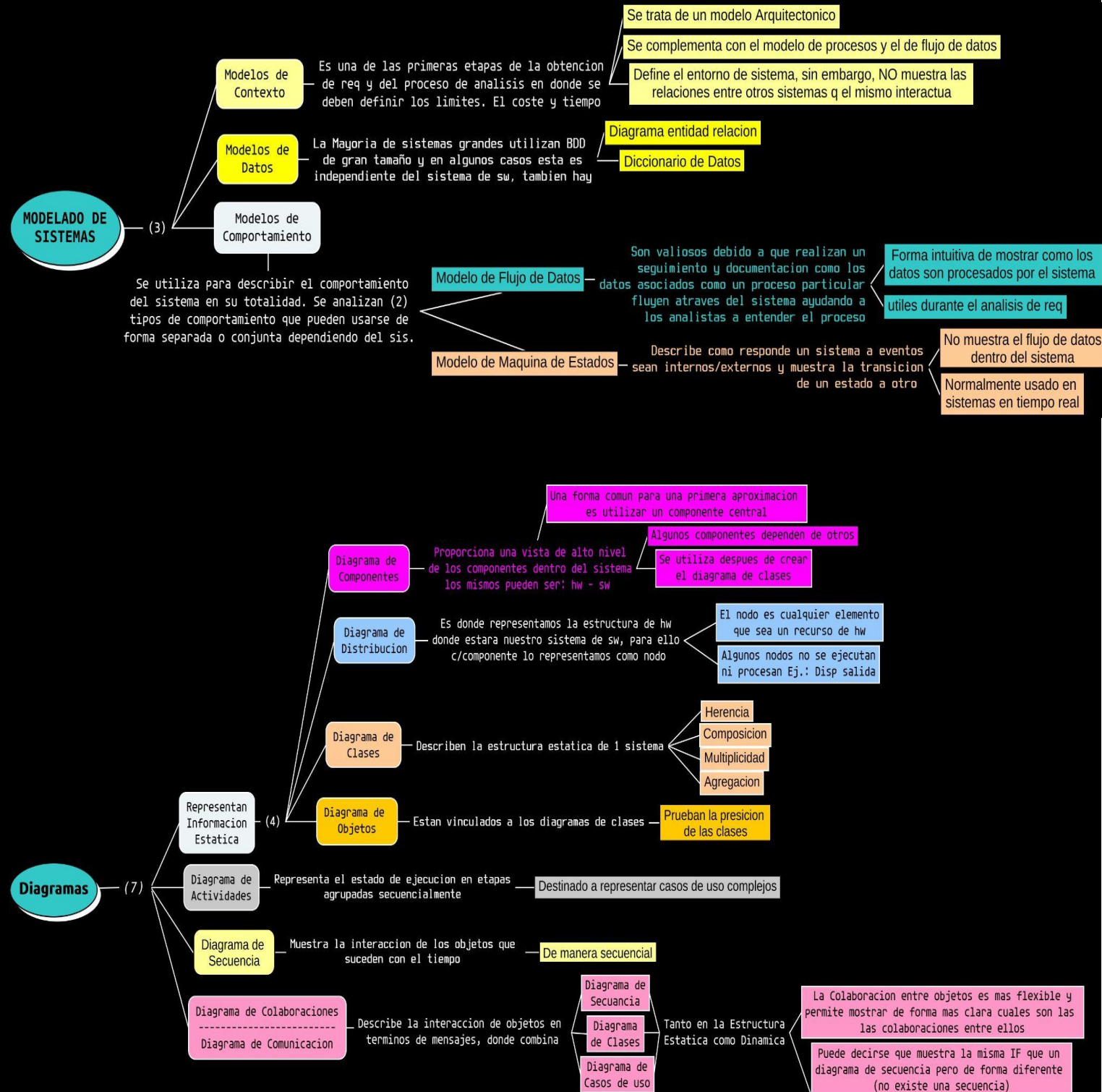
- ☐ **Trazabilidad Bidireccional:** Contar con un mecanismo integral de trazabilidad facilita la finalización de las tareas relacionadas con pruebas y documentación.
- ☐ **Trazabilidad Vertical:** Garantiza que todos los requerimientos sean diseñados, y que todos los diseños se codifiquen y prueben. El mecanismo de trazabilidad no sólo resalta las pruebas que se deben actualizar o repetir sino que también señala los documentos que se deben revisar.
- ☐ **Horizontal:** Es la habilidad de relacionar las secciones/componentes de la misma fase entre sí e identificar las dependencias que haya entre ellas, dependencia/relación de un componente/clase con otros componentes/clases. Permite fácilmente detectar si hay conflictos entre requerimientos, diseño, lógica de codificación o casos de prueba. Cuando se genera un control de cambios sobre un requerimiento, la trazabilidad horizontal brinda la opción de identificar si tendrá un impacto sobre algún otro aspecto de los requerimientos.

IF = información - ing = ingeniería del software - sw = software req = requerimientos - hw = hardware

Tema V - Modelado del Sistema

Los req del usuario deben redactarse en lenguaje natural debido a que tienen que ser comprendido por usuarios NO técnicos, Sin embargo los req del sistema pueden ser detallados de una forma más técnica, Siendo ciertamente una de esas técnicas los modelos de sistemas usada para documentar la especificación del mismo. Estos Diagramas suelen ser a menudo más comprensibles que el lenguaje natural.

- Un Modelo del sistema es una abstracción simplificada de lo que se está estudiando en donde se desarrollan diferentes perspectivas y ciertamente estos modelos se basan en distintas aproximaciones de abstracción.



Tema VI - Otros Métodos de la Ing del Software

Métodos Formales para la especificación y Diseño de Sistemas:

El método formal es cualquier técnica que trate la construcción o el análisis de modelos matemáticos que contribuyen a la automatización del desarrollo de sistemas informáticos.

Se basan en el empleo de técnicas, lenguajes y herramientas definidos matemáticamente para cumplir objetivos tales como facilitar el análisis y construcción de sistemas confiables independientemente de su complejidad delatando posibles ambigüedades que de otra forma podrían pasar inadvertidas.

Sin embargo los detractores aseguran que el empleo de métodos formales supone un volumen de trabajo considerable, aumento en los costes y tiempo de desarrollo que debe quedar supeditado a herramientas que lo automaticen.

Algunas Desventajas:

- La falta de madurez en la práctica del mismo es la causa de la imposibilidad de utilizarlos a nivel industrial tal y como se utilizan otros métodos más comunes en la ing.
- Se considera que la aplicación del mismo encarece los productos y ralentiza su desarrollo.
- Esta especificación es más difícil de aprender que otras técnicas informales

Clasificación de los Métodos Formales (2):

Se han utilizado 2 aproximaciones fundamentales para redactar especificaciones detalladas para sistemas de sw industriales. Estas son las siguientes.

1. Una Aproximación **Algebraica**: En la que el sistema se describe en función de las operaciones y las relaciones. Fue diseñada originalmente para la definición de interfaces de tipo abstracto de datos.
2. Una Aproximación **basada en Modelos**: Se construye un modelo de sistema utilizado en construcciones matemáticas como conjuntos y sucesiones y las operaciones. Ej.: Z, VDM y B

Lenguaje Z:

Las especificaciones se basan en la teoría de conjuntos y en la lógica de primer orden. Z Proporciona una estructura denominada esquema, para describir el estado y las operaciones de una especificación.

El futuro de los métodos Formales - Conclusión:

Aun cuando estas técnicas no se usan con demasiada frecuencia en la industria, ofrecen ciertas ventajas sustanciales, a diferencia de las informales estas pueden estudiarse matemáticamente y ayudan a poder descubrir problemas en los req que podrían ser costosos corregirlos a futuro. De esta manera reduce costes de especificación e implementación eliminando ambigüedad y ofrece un mayor rigor en la primera fase del proceso de ing. Cabe recalcar que existen otras partes del problema que se especifican mejor usando técnicas gráficas.

Ingeniería de Software de Sala Limpia:

Es una filosofía usada en la ing que consiste en evitar las dependencias de los costosos procesos de eliminación de defectos desde el primer momento, por ese motivo se lo denomina como un enfoque limpio(sin errores).

El enfoque hace uso de una versión especializada del modelo incremental del sw. A medida que se va certificando cada incremento, se integra en el todo. inherente a la funcionalidad del sistema que va creciendo con el tiempo. Los certificados en cada incremento son certificaciones estadísticas de calidad a medida que van siendo añadidos al sistema.

IF = información - ing = ingeniería del software - sw = software req = requerimientos - hw = hardware

Tareas que se realizan (8):

Planificación de Incrementos:

Se desarrolla un plan de proyecto que adopta la estrategia incremental. Se van estableciendo las funcionalidades de cada uno de los incrementos, su tamaño estimado y un plan de desarrollo de sala limpia. Es preciso tener especial cuidado para asegurar que los incrementos certificados se vayan integrando de forma temporalmente oportuna.

Recolección de Requisitos:

Mediante el uso de técnicas, se desarrolla una descripción más detallada de requisitos del nivel del usuario (para cada incremento).

Especificación de la Estructura de Cajas:

Se utiliza un método de especificación que hace uso de estructuras de caja para describir la especificación funcional. Ajustado a los principios de análisis operacional, la estructura de caja «aísla y separa la definición creativa del comportamiento, de los datos, y de los procedimientos para cada nivel de refinamiento

- Caja negra. Esta caja especifica el comportamiento del sistema, o de parte de un sistema. El sistema (o parte de él) responde a estímulos específicos (sucesos) mediante la aplicación de un conjunto de reglas de transición que hace corresponder el estímulo con la respuesta.
- Caja de estado. Esta caja encapsula los datos de estados y de servicios (operaciones) de forma análoga a los objetos. En esta vista de especificación, se representan las entradas de la caja de estados (los estímulos) y sus salidas (las respuestas). La caja de estados también representa la «historia de estímulos» de la caja negra, es decir, los datos encapsulados en la caja de estado que deben ser mantenidos entre las transiciones implicadas
- Caja limpia. Las funciones de transición que están implicadas en la caja de estado se definen en la caja limpia. Dicho literalmente, la caja limpia contiene el diseño procedimental correspondiente a la caja de estados.

Diseño formal:

Mediante el uso del enfoque de estructura de cajas, el diseño de sala limpia es una extensión natural y sin discontinuidades de la especificación.

Verificación de corrección:

El equipo de sala limpia lleva a cabo una serie de rigurosas actividades de verificación de corrección aplicadas primero al diseño y después al código.

Generación de código, inspección y verificación:

Las especificaciones de estructura de caja, que se representan mediante un lenguaje especializado, se traducen al lenguaje de programación adecuado. Se utilizan entonces técnicas estándar de recorrido o de inspección para asegurar el cumplimiento semántico de las estructuras de código y de cajas, y la corrección sintáctica de código.

Planificación de la comprobación estadística:

La utilización estimada del software se analiza, se planifica y se diseña un conjunto de casos de prueba que ejerciten la «distribución de probabilidad» de esa utilización, esta actividad de sala limpia se realiza en paralelo con la especificación, la verificación y la generación de código.

Comprobación estadística de utilización:

Las técnicas estadísticas de utilización ejecutan una colección de pruebas derivadas de una muestra estadística de todas las posibles ejecuciones del programa por parte de todos los usuarios de una cierta población objetivo.

Certificación:

Una vez que se ha finalizado la verificación, la inspección y la comprobación de utilización (y después de corregir todos los errores) se certifica el incremento como preparado para su integración

Conclusiones: Hoy en día no es muy usado porque la creencia es más teórica que práctica, y normalmente los científicos que estudian este modelo y trabajan con los datos no conocen el mercado del sw o no están muy familiarizados. También son muy costosos aplicarlos tendrían que ver si el proyecto valdría la pena su uso.

IF = información - ing = ingeniería del software - sw = software req = requerimientos - hw = hardware

Introducción a las características, requerimientos y modelado de aplicaciones para la web (WEB-APPS)

La ingeniería web es la aplicación de metodologías sistemáticas, disciplinadas y cuantificables al desarrollo eficiente, operación y evolución de aplicaciones para la web.

Internet se volvió un desafío para los ing, a raíz de esto se crearon enfoques disciplinados, sistemáticos y metodologías donde tuvieron en cuenta aspectos específicos de este nuevo medio para crear sistemas de sw grandes y complejos.

Metodologías para el desarrollo de web-apps:

1. WSDM: Web site desing method
2. SOHDM: Scenario-Based Object-Oriented Hypermedia Desing Methodology
3. RNA: Relationship Navigational Analysis
4. HFPM: Hypermedia Flexible Process Modeling
5. OOHDM: Object Oriented Hypermedia Design Model
6. UWE: UML-Based Web Engineering
7. W2000
8. UWA: Ubiquitous Web Applications
9. NDT -Navigational Development Techniques

Luego en la teoría da un cuadro comparativo de cada metodología y se puede apreciar que NDT es el que mejor cuadra en cuanto a requisitos.

Aplicación de NDT:

NDT es una técnica para especificar, analizar y diseñar el aspecto de la navegación en aplicaciones web que garantiza la calidad de los resultados. El mismo se centra en las fases de req y análisis y ofrece una guía sistemática para tratar estas fases complementado también con los aspectos de navegación e interfaz abstracta. Ofreciendo una guía con todos los pasos que hay que realizar para tratar los req y a partir de ello conseguir los modelos de análisis.

Diferencia entre el formato IEEE 830 y el Documento NDT

La gran diferencia es que en el desarrollo web no se debe asumir el rol de usuarios estándar ya que la naturaleza de la web es multicultural, además que cada usuario cuenta con capacidades y tecnologías distinta

Ciclo de vida de NDT -> Está compuesto por 2 fases

Ingeniería de requisitos	El Análisis	
Se definen como primera etapa los objetivos del sistema a desarrollar, en base a ello se capturan y definen los req en grupos. Cada grupo es tratado de manera particular, adecuada a su tipología. Si durante la validación se encuentran errores se usa backtracking	Durante esta etapa se generan 2 Modelos	
	Conceptual	Navegación
	Representa la estructura estática y viene acompañado por un diagrama de clases	Compone una serie de diagramas, con notación muy similar al diagrama de clases

La Aplicación NDT genera 3 resultados finales

- El documento de requisitos del sistema, donde se detallan los objetivos y req que se debe cumplir
- El documento de análisis donde recoge el modelo conceptual y el modelo de navegación
- Los prototipos del sistema, que demuestran la estructura de la interfaz abstracta del sistema.

Sistemas en Tiempo Real:

Una de las áreas de expansión más rápidas de la explotación de computadores es aquella que implica aplicaciones cuya función principal no es la de procesar IF, sin embargo necesita de dicho proceso con el fin de realizar su función principal. Y justamente a este tipo de aplicaciones se les conoce generalmente como sistemas en tiempo real o sistemas embebidos.

Cualquier actividad o sistema de proceso de IF que tenga que responder a un estímulo de entrada generado externamente en periodo finito y especificado. (Young 1982)

Estos sistemas si bien ya es obvio se diferencian de otros tipos debido a que su correcto funcionamiento depende de que el sistema responda a los eventos dentro de un corto intervalo de tiempo. En la teoría clasifica el tiempo de respuesta en estricto (imperativo) e NO estricto donde los tiempos tal vez no se cumplan

Características de los Sistemas en tiempo real:

1. **Grande y complejo**

Los programas grandes como todos los sistemas complejos, deben evolucionar continuamente.

2. **Manipulación de números reales**

Un req funcional de un lenguaje de programación en tiempo real es, por tanto la capacidad para manipular números reales o de coma flotante. Para que el computador pueda resolver ecuaciones diferenciales mediante algoritmos y técnicas numéricas con el fin de conseguir los valores de entrada requeridos.

3. **Fiabiles y seguros**

Donde se precise interacción con un operador, deberá tenerse especial cuidado en el diseño de la interfaz, para el fin de minimizar la posibilidad de error humano, y no solo deben tenerse en cuenta estas dificultades inherentes sino también por un diseño de sw defectuoso.

4. **Control concurrente de los distintos componentes separados del sistema**

Suele constar de varios elementos con los que los programas deben interactuar simultáneamente, Agregar lo del libro

5. **Funcionalidades en tiempo real**

El tiempo de respuesta es crucial en cualquier sistema embebido, por ese motivo se construyen habitualmente con procesadores con capacidades considerables, garantizando la minimización del retraso en cuanto al tiempo de respuesta.

6. **Interacción con interfaces de hardware**

Se necesita para monitorizar sensores y controlar actuadores para una amplia variedad de dispositivos en tiempo real, en el pasado las realizaba bajo supervisión del SO, actualmente su control es directo debido a las técnicas de bajo nivel de los sistemas embebidos

7. **Implementación eficiente y entorno de ejecución**

Puesto que los sistemas de tiempo real son críticos respecto al tiempo, la eficiencia de la implementación será más importante que en otros sistemas

IF = información - ing = ingeniería del software - sw = software req = requerimientos - hw = hardware

Sistemas de Telefonía Móvil

Una aplicación móvil consiste en un sw que funciona en un dispositivo móvil y ejecuta ciertas tareas para el usuario. En el mundo del marketing móvil es el que mayor crecimiento ha experimentado en los últimos años y el concepto de tienda de aplicaciones que ofrecen las distintas marcas hasta hoy en día el sw libre.

Y la revolución del Mercado con la creación de tabletas para usar como cliente, en sistemas de sw grandes.

Categoría de las Aplicaciones:

1. Donde Funciona (Según entorno de ejecución)

Las 2 más usadas hoy en día es en los SO móviles nativos y en aplicaciones de Ingeniería web, aunque esta última tiene un grado de aprovechamiento menor.

2. En Base a funcionalidades que aporta al usuario

Conjunto de sistemas embebidos que conforman la aplicación.

Análisis y definición de Objetivos:

Las aplicaciones especializadas para empresas pueden ser excelentes herramientas de gestión y de procesos de trabajo. Por ejemplo: whatsApp for Business

También como herramienta y canal de venta (mobile e-commerce) que puede consistir en la venta de productos o servicios para una empresa. Por último también para generar ingresos, ejemplo: bitcoin

Hablar de la comercialización de apple y sus productos para desarrollar sw

FRAGMENTACIÓN:

Es una situación en la que no es posible compartir una misma aplicación entre diferentes ecosistemas. Es decir impide que se pueda compartir la aplicación sin adaptar ecosistemas.

Puede originarse por los siguientes motivos.

- hw diferente: Componentes distintos, osea de distintas marcas
- sw diferente: por ejemplo el sistema operativo
- variaciones de la funcionalidades:
- preferencias del usuario : como idiomas y orientación

Problemas que se presentan al NO tratar la fragmentación

- Reducir la calidad del producto. Debido a la mayor complejidad de las soluciones fragmentadas, se pueden generar más errores
- Limitar el número de dispositivos soportados. Para empezar, luego a futuro se podría ampliar
- Alargar cualquier fase del proyecto. Esta dilatación supondrá un coste mayor al acordado, así como posible fracaso de dicho proyecto.

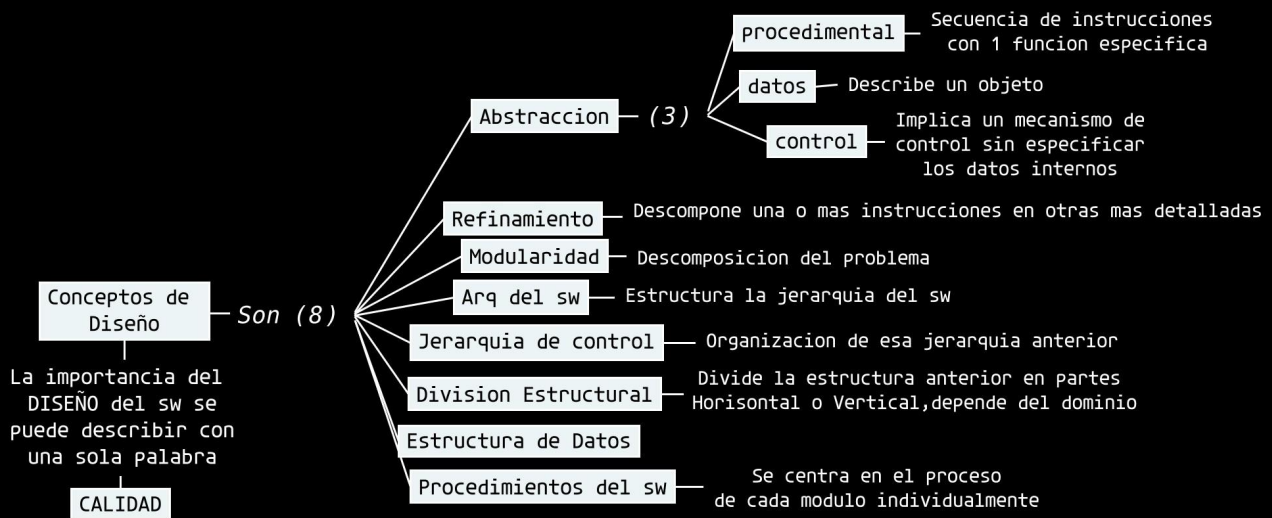
Sin duda, la fragmentación ha sido y seguramente será, la mayor dificultad y el mayor riesgo en el desarrollo de aplicaciones.

Tema VII - Ingeniería de Diseño y Testeo

Introducción al Diseño de Componentes:

Transforma los elementos estructurales de la Arquitectura del sw en una descripción procedimental de los componentes del sw. Esta consta de (4) capas.

- 1) Capa de subsistema: Representación de c/uno de los subsistemas para permitir al sw conseguir sus req e implementar la infraestructura que la soporte.
- 2) Capa de Clases y Objetos: Jerarquía de clases que permiten al sistema usar generalidades y especificaciones.
- 3) Capa de Mensajes: Contiene detalles de diseño que permite a cada objeto comunicarse con sus colaboradores.
- 4) Capa de Responsabilidades: Contiene estructuras de datos y diseños algorítmicos, para todos los atributos y comportamiento de cada objeto.



Alta cohesión y Bajo Acoplamiento:

La independencia funcional es = modularidad + conceptos de abstracción y ocultación de la IF. Esa independencia mencionada se mide mediante 2 criterios cualitativos.

Cohesión: Lleva a cabo **una sola tarea** en el procedimiento del sw, lo cual requiere poca interacción con los procedimientos usados en otras partes del mismo programa.

Acoplamiento: Medida de interconexión entre módulos dentro de 1 estructura de sw, el mismo dependerá de la complejidad de interconexión entre los módulos.

Introducción al Diseño de las Bases de Datos:

Consiste en definir la estructura de los datos que debe tener la BD de un sistema de IF. En el caso relacional será un conjunto de esquemas.

Etapas del Diseño de las bases de datos:

Diseño Conceptual: Se obtiene una estructura de la IF de la futura BD independientemente de la tecnología que hay que emplear. No se tiene en cuenta que tipo de BD se utilizara ni tampoco el lenguaje que se usará, se apoyará mediante el modelo entidad relación. Junto con la normalización etc.

Diseño Lógico: Parte del resultado del diseño conceptual, se encuadra tanto el lenguaje con el tipo de BD y permite concentrarse en cuestiones tecnológicas relacionadas con el modelo de base de datos.

Diseño Físico: La diferencia con la etapa anterior es que ahora se tiene en cuenta aspectos de carácter más físicos del mundo de las representaciones con el objetivo de conseguir una mayor eficiencia.